CSC336 Assignment 1:Numerical Methods

Hongcheng Huo 1009801626

January 2025

Question 1: Floating-Point Summation by round-to-nearest

We are given the sum:

$$S = 801 + 150 + 72 + 14 + 4 + 3 + 3 + 3$$

we must evaluate it using the three-decimal digit floating-point system $F(10, 3, -10, 10)$ with round-to-nearest. The summation must be performed in two different orders:

- Left-to-right: Start from 801 and add the next numbers sequentially.
- Right-to-left: Start from 3 and add the next numbers in reverse order.

Floating-Point Representation

In $F(10, 3, -10, 10)$, each number must be written in normalized scientific notation with 3 significant digits:

$$\begin{aligned}
801 &= 8.01 \times 10^2 \\
150 &= 1.50 \times 10^2 \\
72 &= 7.20 \times 10^1 \\
14 &= 1.40 \times 10^1 \\
4 &= 4.00 \times 10^0 \\
3 &= 3.00 \times 10^0
\end{aligned}$$

Left-to-Right Summation

Performing the sum step by step:

$$\begin{aligned}
(8.01 \times 10^2) + (1.50 \times 10^2) &= 9.51 \times 10^2 \quad \text{(rounded)} \\
(9.51 \times 10^2) + (7.20 \times 10^1) &= 1.02 \times 10^3 \quad \text{(rounded)} \\
(1.02 \times 10^3) + (1.40 \times 10^1) &= 1.03 \times 10^3 \quad \text{(rounded)} \\
(1.03 \times 10^3) + (4.00 \times 10^0) &= 1.03 \times 10^3 \quad \text{(rounded)} \\
(1.03 \times 10^3) + (3.00 \times 10^0) &= 1.03 \times 10^3 \quad \text{(rounded)} \\
(1.03 \times 10^3) + (3.00 \times 10^0) &= 1.03 \times 10^3 \quad \text{(rounded)} \\
(1.03 \times 10^3) + (3.00 \times 10^0) &= 1.03 \times 10^3 \quad \text{(rounded)}
\end{aligned}$$

Thus, the final left-to-right floating-point sum is:

$$S_{\text{left-to-right}} = 1.03 \times 10^3 = 1030$$

Right-to-Left Summation

Performing the sum step by step in reverse order:

$$
\begin{aligned}
(3.00 \times 10^0) + (3.00 \times 10^0) &= 6.00 \times 10^0 \quad \text{(rounded)} \\
(6.00 \times 10^0) + (3.00 \times 10^0) &= 9.00 \times 10^0 \quad \text{(rounded)} \\
(9.00 \times 10^0) + (4.00 \times 10^0) &= 1.30 \times 10^1 \quad \text{(rounded)} \\
(1.30 \times 10^1) + (1.40 \times 10^1) &= 2.70 \times 10^1 \quad \text{(rounded)} \\
(2.70 \times 10^1) + (7.20 \times 10^1) &= 9.90 \times 10^1 \quad \text{(rounded)} \\
(9.90 \times 10^1) + (1.50 \times 10^2) &= 2.49 \times 10^2 \quad \text{(rounded)} \\
(2.49 \times 10^2) + (8.01 \times 10^2) &= 1.05 \times 10^3 \quad \text{(rounded)}
\end{aligned}
$$

Thus, the final right-to-left floating-point sum is:

$$S_{\text{right-to-left}} = 1.05 \times 10^3 = 1050$$

Exact Summation

The exact sum is:

$$S_{\text{exact}} = 801 + 150 + 72 + 14 + 4 + 3 + 3 + 3 = 1050$$

Relative Error Analysis

Now, we compute the relative errors:

$$\text{Relative Error} = \frac{|S_{\text{computed}} - S_{\text{exact}}|}{S_{\text{exact}}}$$

For left-to-right summation:

$$\frac{|1030 - 1050|}{1050} = \frac{20}{1050} \approx 0.0190 \quad (1.90\%)$$

For right-to-left summation:

$$\frac{|1050 - 1050|}{1050} = 0.0 \quad (0\%)$$

Conclusion

This experiment demonstrates the impact of floating point arithmetic on the order of summation.

- **Floating-point arithmetic is not associative**: The order of operations affects the final result due to rounding errors.
- **Accumulated rounding error**: In left-to-right summation, larger numbers are added first, which can cause smaller numbers to lose precision when added later.
- **Summing smaller numbers first reduces error**: Right-to-left summation tends to be more accurate because smaller numbers are added first, minimizing rounding errors and preserving accuracy.
- **Final Conclusion:** When performing floating-point summations, it is generally better to sum smaller numbers first to reduce the accumulated rounding errors and improve numerical stability.

Question 2: Floating-Point System

Question 2(a): Counting Number of Floating Ponit

IEEE 754 Double Precision Floating-Point Representation

- 1-bit sign
- 11-bit exponent
- 52-bit mantissa

The floating-point number x can be represented as:

$$x = (-1)^s \times 2^{(e-1023)} \times (1 + m \times 2^{-52})$$

For normalized floating-point numbers, when $e = 1023$, we have:

$$1 \leq x < 2$$

Thus x takes values of:

$$x = 1 + m \times 2^{-52}, \quad 0 \leq m \leq 2^{52} - 1$$

Hence, the total number of floating-point numbers in the interval $1 \leq x < 2$ is:

$$2^{52}$$

Counting Floating-Point Numbers in $1 \leq x < 2$

All numbers satisfying $1 \leq x < 2$ are given by:

$$x = 1 + m \times 2^{-52}, \quad 0 \leq m \leq 2^{52} - 1$$

Since m ranges from 0 to $2^{52} - 1$, the total count is:

$$2^{52}$$

Counting Floating-Point Numbers in $1 \leq x < \frac{3}{2}$

- Substituting $x = 1 + m \times 2^{-52}$:

$$0 \leq m \times 2^{-52} < \frac{1}{2}$$

- Multiplying both sides by 2^{52}:

$$0 \leq m < 2^{51}$$

- Thus the number of floating-point values in $1 \leq x < \frac{3}{2}$ is:

$$2^{51}$$

Counting numbers in $\frac{3}{2} \leq x < 2$

- Substituting $x = 1 + m \times 2^{-52}$:

$$\frac{1}{2} \leq m \times 2^{-52} < 1$$

- Multiplying both sides by 2^{52}:

$$2^{51} \leq m < 2^{52}$$

- Thus the number of floating-point values in $\frac{3}{2} \leq x < 2$ is:

$$2^{51}$$

Question 2(b): Counting Number of Floating Point

Counting Floating-Point Numbers in $\frac{1}{2} \leq y < 1$

All numbers satisfying $\frac{1}{2} \leq y < 1$ are given by:

$$y = \frac{1}{2} + m \times 2^{-53}, \quad 0 \leq m \leq 2^{52} - 1$$

Since m ranges from 0 to $2^{52} - 1$, the total count is:

$$2^{52}$$

Counting numbers in $\frac{1}{2} \leq y < \frac{2}{3}$

- Substituting $y = \frac{1}{2} + m \times 2^{-53}$:

$$0 \leq m \times 2^{-53} < \frac{2}{3} - \frac{1}{2}$$

$$0 \leq m \times 2^{-53} < \frac{1}{6}$$

- Multiplying both sides by 2^{53}:

$$0 \leq m < \frac{1}{6} \times 2^{53}$$

$$0 \leq m < \frac{2^{53}}{6} = 2^{50} \times \frac{2}{3}$$

- Thus the number of floating-point values in $\frac{1}{2} \leq y < \frac{2}{3}$ is:

$$\frac{2^{52}}{3}$$

Counting numbers in $\frac{2}{3} \leq y < 1$

- Substituting $y = \frac{1}{2} + m \times 2^{-53}$:

$$\frac{1}{6} \leq m \times 2^{-53} < \frac{1}{2}$$

- Multiplying both sides by 2^{53}:

$$\frac{2^{53}}{6} \leq m < 2^{52}$$

- Computing:

$$\text{Floating-point count} = 2^{52} - \frac{2^{53}}{6} = \frac{2^{52} \times 2}{3}$$

$$\frac{2^{52} \times 2}{3}$$

Question 2(c): Existence of Floating-Point Numbers

Proof of Existence

IEEE 754 double precision floating-point numbers are finite and discrete, whereas the reciprocal function is continuous and strictly decreasing in $1 \leq x < 2$.

$$f(x) = \frac{1}{x}$$

Since:

- The number of floating-point values in $1 \leq x < 2$ is 2^{52}.
- The number of floating-point values in $\frac{1}{2} \leq y < 1$ (where $y = 1/x$) is also 2^{52}.
- The function $f(x) = 1/x$ maps $1 \leq x < 2$ onto $\frac{1}{2} \leq y < 1$.

it follows that, due to finite precision and rounding, at least two different values x_1, x_2 must exist such that:

$$\text{round}\left(\frac{1}{x_1}\right) = \text{round}\left(\frac{1}{x_2}\right).$$

Occourrence Interval

Floating-point distribution effects:

- In $1 \leq x < \frac{3}{2}$, floating-point numbers are densely packed.
- In $\frac{3}{2} \leq x < 2$, floating-point spacing is wider.
- Since $f(x) = 1/x$ changes more rapidly for smaller x, rounding errors are more significant.

Thus, it is more likely that two distinct numbers x_1, x_2 will be mapped to the same rounded reciprocal in the interval:

$$1 \leq x < \frac{3}{2}.$$

Question 2(d)

Analysis of $(1/x) \times x \neq 1$

In IEEE 754 double precision, multiplication is not always exact due to finite precision and rounding errors. The computation:

$$y = \frac{1}{x}$$

involves a division, and:

$$z = y \times x$$

involves a multiplication. If both operations introduce rounding errors, we may have:

$$z \neq 1.$$

main reasons:

- If x is not exactly representable in base 2, then $1/x$ cannot be represented exactly.
- When multiplying $(1/x) \times x$, rounding occurs, leading to small deviations from 1.

Thus, there exist floating-point numbers x such that:

$$(1/x) \times x \neq 1.$$

Analysis of $1/(1/x) \neq x$

The computation:

$$y = \frac{1}{x}, \quad z = \frac{1}{y}$$

involves two floating-point divisions. If rounding occurs in computing y, then z will not necessarily equal x.

main reasons:

- The operation $1/x$ introduces rounding errors.
- The operation $1/(1/x)$ propagates and amplifies these errors.
- If x is not exactly representable in IEEE 754, then rounding will cause $1/(1/x) \neq x$.

Thus, there exist floating-point numbers x such that:

$$1/(1/x) \neq x.$$

Question 2e

Error Analysis of $1/(1/x)$

In IEEE 754 double precision:

- x is stored with 52 bits of precision.
- Computing $1/x$ introduces rounding error.
- Computing $1/(1/x)$ propagates the error.

If x is exactly representable in binary, then $1/x$ and $1/(1/x)$ can be computed without error. However, most values in $3/2 \leq x < 2$ are not exactly representable.

Estimation of the Fraction

- The total number of floating-point numbers in $\frac{3}{2} \leq x < 2$ is 2^{51}.
- The subset for which x is exactly represented consists of numbers where the binary representation.
- Around 2^{-52} of the numbers in $3/2 \leq x < 2$ are exactly representable.

Thus, the approximate fraction is:

$$\frac{1}{2^{52}} \approx 2.22 \times 10^{-16}.$$

Question 2f

Python Implementation

```
import random

# Number of samples
num_samples = 1000000
count_exact = 0

# Generate random samples in the interval [1.5, 2.0)
for _ in range(num_samples):
    x = random.uniform(1.5, 2.0)
    if 1 / (1 / x) == x:
        count_exact += 1

# Compute fraction
fraction = count_exact / num_samples
print(f"Fraction of cases where 1/(1/x) == x: {fraction:.16f}")
```

Explanation of the Code

- generate 10^6 random floating-point numbers x in the interval $[1.5, 2.0)$.
- compute $1/(1/x)$ and check if it exactly equals x.
- The fraction of cases where the equality holds is computed.

Expected Result

```
>>> print(f"Fraction of cases where 1/(1/x) == x: {fraction:.16f}")
Fraction of cases where 1/(1/x) == x: 0.6668430000000000
```

Explanation

- Unlike the theoretical estimate of 2.22×10^{-16}, around 66.67% of sampled numbers satisfy $1/(1/x) = x$.
- Many numbers are still exactly representable after taking the reciprocal twice.
- However, around 33.33% of numbers experience floating-point rounding errors.

Question 3: Optimization of Archimedean Polygon Approximation

Original Recursive Formula

The original Archimedean recursive formula for computing the perimeter of an inscribed polygon with 2^n sides is:

$$p_{n+1} = 2^n \sqrt{2(1 - \sqrt{1 - (p_n/2^n)^2})}$$

Original Algorithm Code

```python
""" Original Archimedean Polygon Approximation Algorithm Code"""
import math
import pandas as pd


1 个用法
def compute_p_n(p_prev, two_power_prev):
    """ Recursive function to compute p_n based on the previous value p_(n-1) """

    # Compute 2^n recursively
    two_power_n = 2 * two_power_prev

    # Compute p_n recursively
    p_n = two_power_prev * math.sqrt(2 * (1 - math.sqrt(1 - (p_prev / two_power_prev) ** 2)))

    # return the p_n and the edges(two_power_n)
    return p_n, two_power_n
```

Figure 1: Part 1

```python
1 个用法
def compute_pi_approximations(n_min=3, n_max=40):
    """ Compute the approximation of π using recursive computation of p_n """

    # storing the results(n, p_n, relative_error)
    results = []

    # Initial conditions
    p_2 = 2 * math.sqrt(2)  # p_2 for square
    two_power_2 = 4  # 2^2 = 4 for square

    # Recursively compute p_n values
    p_prev, two_power_prev = p_2, two_power_2

    for n in range(n_min, n_max + 1):
        p_n, two_power_n = compute_p_n(p_prev, two_power_prev)

        # Compute relative error
        relative_error = abs(math.pi - p_n) / math.pi

        # Store results
        results.append((n, f"{p_n:.15f}", f"{relative_error:.3e}"))

        # Update previous values for next iteration
        p_prev, two_power_prev = p_n, two_power_n

    return results

```

Figure 2: Part 2

```python
def display_results(results):
    """ Display the computed approximations in a tabular format using pandas """
    df = pd.DataFrame(results, columns=["n", "p_n (15 decimal digits)", "Relative Error (3 decimal digits)"])
    print(df.to_string(index=False))


1 个用法
def main():
    """ execute the algorithm """
    results = compute_pi_approximations()
    display_results(results)


if __name__ == "__main__":
    main()

```

Figure 3: Part 3

Original Output Data

n	p_n (15 decimal digits)	Relative Error (3 decimal digits)
3	3.061467458920719	2.550e-02
4	3.121445152258053	6.413e-03
5	3.136548490545941	1.606e-03
6	3.140331156954739	4.015e-04
7	3.141277250932757	1.004e-04
8	3.141513801144145	2.510e-05
9	3.141572940367883	6.275e-06
10	3.141587725279961	1.569e-06
11	3.141591421504635	3.922e-07
12	3.141592345611077	9.803e-08
13	3.141592576545004	2.452e-08
14	3.141592633463248	6.406e-09
15	3.141592654807589	3.876e-10
16	3.141592645321215	2.632e-09
17	3.141592607375720	1.471e-08
18	3.141592910939673	8.192e-08
19	3.141594125195191	4.684e-07
20	3.141596553704820	1.241e-06
21	3.141596553704820	1.241e-06
22	3.141674265021758	2.598e-05
23	3.141829681889202	7.545e-05
24	3.142451272494134	2.733e-04
25	3.142451272494134	2.733e-04
26	3.162277660168380	6.584e-03
27	3.162277660168380	6.584e-03
28	3.464101615137754	1.027e-01
29	4.000000000000000	2.732e-01
30	0.000000000000000	1.000e+00

Figure 4: Part 4

30	0.000000000000000	1.000e+00
31	0.000000000000000	1.000e+00
32	0.000000000000000	1.000e+00
33	0.000000000000000	1.000e+00
34	0.000000000000000	1.000e+00
35	0.000000000000000	1.000e+00
36	0.000000000000000	1.000e+00
37	0.000000000000000	1.000e+00
38	0.000000000000000	1.000e+00
39	0.000000000000000	1.000e+00
40	0.000000000000000	1.000e+00

Figure 5: Part 5

Experimental Data Analysis

To understand the behavior data of the recursive formula for computing the perimeter of inscribed polygons, we analyze the results in three distinct phases:

Phase 1: Convergence to π ($n = 3$ to $n = 15$)

During this phase, the computed values of p_n rapidly converge to π, and the relative error decreases exponentially.

n	p_n (15 decimal digits)	Relative Error
3	3.061467458920719	2.550×10^{-2}
6	3.140331156954753	4.015×10^{-4}
10	3.141587725277160	1.569×10^{-6}
15	3.141592648776985	1.532×10^{-9}

Table 1: Phase 1: Initial convergence to π

Observations:

- The computed values approach π with high accuracy.
- The relative error decreases exponentially as n increases.
- The algorithm is numerically stable in this range.

Phase 2: Increasing Numerical Error ($n = 16$ to $n = 29$)

In this phase, the relative error stops decreasing and begins to increase due to floating-point precision limitations.

n	p_n (15 decimal digits)	Relative Error
16	3.141592645321215	2.632×10^{-9}
20	3.141596553704820	1.241×10^{-6}
25	3.142451272494134	2.733×10^{-4}
29	4.000000000000000	2.732×10^{-1}

Table 2: Phase 2: Numerical error increases

Observations:

- Initially, the error appears to stabilize at 10^{-9}.
- However, beyond $n = 16$, the error begins to grow.
- At $n = 29$, the computed value of p_n deviates significantly from π.
- This suggests that floating-point precision issues are affecting the computation.

Phase 3: Computation Breakdown ($n = 30$ to $n = 40$)

In this phase, the computed values collapse to zero, and the relative error reaches 100%.

n	p_n (15 decimal digits)	Relative Error
30	0.000000000000000	1.000
35	0.000000000000000	1.000
40	0.000000000000000	1.000

Table 3: Phase 3: Complete numerical failure

Observations: (next page)

- At $n = 30$, the computed value of p_n drops to zero.
- All subsequent values remain zero, indicating a complete breakdown.
- The relative error reaches 100%.
- This suggests that floating-point underflow has occurred, making further computation meaningless.

Sources of Numerical Errors

The observed numerical errors can be attributed to the following key factors (I guess):

1. Floating-Point Rounding Error

Floating-point arithmetic in IEEE 754 double precision has a finite precision of approximately 10^{-16}. As n increases:

- The term $\left(\frac{p_n}{2^n}\right)^2$ becomes very small.
- The computation of $\sqrt{1-x}$ may suffer from rounding errors when $x \approx 0$.

2. Cancellation Error

When computing:

$$1 - \sqrt{1-x}$$

for small x, significant precision loss occurs due to subtraction of nearly equal numbers, leading to inaccurate results.

3. Floating-Point Underflow

- As n increases, $\frac{p_n}{2^n}$ approaches a very small value:

$$\frac{p_n}{2^n} \approx \frac{\pi}{2^n}$$

- For large n, this value becomes smaller than the smallest representable floating-point number, causing it to be rounded to zero. This results in:

$$\sqrt{1-(p_n/2^n)^2} \approx 1$$

  which eventually leads to $p_n = 0$, causing a complete breakdown.

4. Error Accumulation

Since p_{n+1} depends on p_n, any error introduced at one step propagates to subsequent steps. As n increases, this accumulated error becomes dominant, leading to numerical instability and incorrect results.

Improved Recursive Formula

- To reduce numerical errors, we use an equivalent transformation:

$$1 - \sqrt{1-x} = \frac{x}{1+\sqrt{1-x}}$$

- Substituting $x = (p_n/2^n)^2$ into the original recurrence formula, we obtain:

$$p_{n+1} = p_n \times \sqrt{\frac{2}{1+\sqrt{1-(p_n/2^n)^2}}}$$

- This modification ensures that p_n remains stable and accurate for larger values of n, preventing the computational breakdown observed in the original formula.

Improved Algorithm Code

```python
""" Improved Archimedean Polygon Approximation Algorithm Code"""
import math
import pandas as pd


1 个用法
def compute_p_n(p_prev, two_power_prev):
    """
    Recursive function to compute p_n based on the previous value p_{n-1}
    using the improved stable recurrence formula.
    """
    # Compute 2^n recursively
    two_power_n = 2 * two_power_prev

    # Compute p_n using the improved formula
    term = (p_prev / two_power_prev) ** 2
    correction_factor = math.sqrt(2 / (1 + math.sqrt(1 - term)))
    p_n = p_prev * correction_factor

    return p_n, two_power_n
```

Figure 6: Part 1

```python
def compute_pi_approximations(n_min=3, n_max=40):
    """ Compute the approximation of π using recursive computation of p_n """
    # Storing the results(n, p_n, relative_error)
    results = []

    # Initial conditions
    p_2 = 2 * math.sqrt(2)  # p_2 for square
    two_power_2 = 4  # 2^2 = 4 for square

    # Recursively compute p_n values
    p_prev, two_power_prev = p_2, two_power_2

    for n in range(n_min, n_max + 1):
        p_n, two_power_n = compute_p_n(p_prev, two_power_prev)

        # Compute relative error
        relative_error = abs(math.pi - p_n) / math.pi

        # Store results
        results.append((n, f"{p_n:.15f}", f"{relative_error:.3e}"))

        # Update previous values for next iteration
        p_prev, two_power_prev = p_n, two_power_n

    return results
```

Figure 7: Part 2

```python
def display_results(results):
    """ Display the computed approximations in a tabular format using pandas """
    df = pd.DataFrame(results, columns=["n", "p_n (15 decimal digits)", "Relative Error (3 decimal digits)"])
    print(df.to_string(index=False))


1 个用法
def main():
    """ execute the algorithm """
    results = compute_pi_approximations()
    display_results(results)


if __name__ == "__main__":
    main()
```

Figure 8: Part 3

Better Approximation Data

n	p_n (15 decimal digits)	Relative Error (3 decimal digits)
3	3.061467458920719	2.550e-02
4	3.121445152258053	6.413e-03
5	3.136548490545940	1.606e-03
6	3.140331156954753	4.015e-04
7	3.141277250932773	1.004e-04
8	3.141513801144301	2.510e-05
9	3.141572940367091	6.275e-06
10	3.141587725277160	1.569e-06
11	3.141591421511200	3.922e-07
12	3.141592345570118	9.805e-08
13	3.141592576584872	2.451e-08
14	3.141592634338563	6.128e-09
15	3.141592648776985	1.532e-09
16	3.141592652386591	3.830e-10
17	3.141592653288992	9.575e-11
18	3.141592653514593	2.394e-11
19	3.141592653570993	5.984e-12
20	3.141592653585093	1.496e-12
21	3.141592653588618	3.740e-13
22	3.141592653589499	9.358e-14
23	3.141592653589719	2.347e-14
24	3.141592653589774	5.937e-15
25	3.141592653589788	1.555e-15
26	3.141592653589792	4.241e-16
27	3.141592653589793	1.414e-16
28	3.141592653589793	1.414e-16
29	3.141592653589793	1.414e-16
30	3.141592653589793	1.414e-16

Figure 9: Part 4

30	3.141592653589793	1.414e-16
31	3.141592653589793	1.414e-16
32	3.141592653589793	1.414e-16
33	3.141592653589793	1.414e-16
34	3.141592653589793	1.414e-16
35	3.141592653589793	1.414e-16
36	3.141592653589793	1.414e-16
37	3.141592653589793	1.414e-16
38	3.141592653589793	1.414e-16
39	3.141592653589793	1.414e-16
40	3.141592653589793	1.414e-16

Figure 10: Part 5

With n becoming larger, the relative error becomes smaller(by observing the exponent part) and the p_n becomes more accurate as it is closer to the irrational number π. (keeping 15 decimal digits)

Question 4: Condition Number Analysis and Algorithm Improvement

Continuity of $f(x)$ at $x = 0$

The function $f(x)$ is defined as:

$$f(x) = \frac{1 - \cos(x)}{\sin(x)}$$

To check the continuity at $x = 0$, we compute the limit:

$$\lim_{x \to 0} f(x) = \lim_{x \to 0} \frac{1 - \cos(x)}{\sin(x)}$$

Since

$$1 - \cos(x) \to 0, \quad \sin(x) \to 0$$

This forms an indeterminate $\frac{0}{0}$ type, so we apply L'Hôpital's rule:

$$\lim_{x \to 0} \frac{1 - \cos(x)}{\sin(x)} = \lim_{x \to 0} \frac{\sin(x)}{\cos(x)}$$

Since

$$\lim_{x \to 0} \tan(x) = 0$$

we conclude

$$\lim_{x \to 0} f(x) = 0$$

Since $f(0) = 0$, we verify:

$$\lim_{x \to 0} f(x) = f(0)$$

Thus $f(x)$ is continuous at $x = 0$.
Further we can also prove that $f(x)$ is continuous on its domain $[-\pi/2, +\pi/2]$

Computation of the Condition Number $\kappa(x)$

The condition number is defined as:

$$\kappa(x) = \left| \frac{x f'(x)}{f(x)} \right|$$

Using the quotient rule on function

$$f'(x) = \frac{(1 - \cos(x))' \sin(x) - (1 - \cos(x))(\sin(x))'}{\sin^2(x)}$$

Rearranging:

$$f'(x) = 1 - \frac{(1 - \cos(x)) \cos(x)}{\sin^2(x)}$$

Finally we obtain:

$$\kappa(x) = \left| \frac{x \left(1 - \frac{(1 - \cos(x)) \cos(x)}{\sin^2(x)}\right)}{\frac{1 - \cos(x)}{\sin(x)}} \right|$$

From numerical evaluation:

- Near $x = 0$, $\kappa(x) \approx 1$, meaning good numerical stability.
- Near $x = \pm\pi/2$, $\kappa(x) \approx 1.57$, still stable.
- The maximum value is $\max \kappa(x) = 1.57$, which does not cause significant error amplification.

Evaluation $\kappa(0)$

- First compute the limits of $f(x)$ as $x \to 0$

  Expanding using Taylor series:

$$1 - \cos(x) \approx \frac{x^2}{2}, \quad \sin(x) \approx x$$

- Substituting these approximations into $f(x)$,

$$f(x) \approx \frac{\frac{x^2}{2}}{x} = \frac{x}{2}$$

- Next compute the Limit of $f'(x)$ at $x = 0$

  Using the approximations:

$$1 - \cos(x) \approx \frac{x^2}{2}, \quad \cos(x) \approx 1, \quad \sin^2(x) \approx x^2$$

- Substituting these approximations into $f'(x)$

$$f'(x) \approx 1 - \frac{\frac{x^2}{2} \cdot 1}{x^2} = 1 - \frac{1}{2} = \frac{1}{2}$$

- Now we evaluate:

$$\kappa(0) = \lim_{x \to 0} \left| \frac{x f'(x)}{f(x)} \right|$$

$$\kappa(0) = \left| \frac{x \cdot \frac{1}{2}}{\frac{x}{2}} \right| = 1$$

- Thus the condition number at $x = 0$ is:

$$\kappa(0) = 1$$

Evaluation $\kappa(+\pi/2)$ and $\kappa(-\pi/2)$

- Computing $f(\pm\pi/2)$

$$f(\pi/2) = \frac{1 - \cos(\pi/2)}{\sin(\pi/2)} = \frac{1 - 0}{1} = 1$$

$$f(-\pi/2) = \frac{1 - \cos(-\pi/2)}{\sin(-\pi/2)} = \frac{1 - 0}{-1} = -1$$

- Computing $f'(\pm\pi/2)$

$$f'(\pi/2) = 1 - \frac{(1 - 0) \cdot 0}{1} = 1$$

$$f'(-\pi/2) = 1 - \frac{(1 - 0) \cdot 0}{1} = 1$$

- Computing $\kappa(\pm\pi/2)$

$$\kappa(\pi/2) = \left| \frac{(\pi/2) \cdot 1}{1} \right| = \frac{\pi}{2}$$

$$\kappa(-\pi/2) = \left| \frac{(-\pi/2) \cdot 1}{-1} \right| = \frac{\pi}{2}$$

Conclusion

- At $x = 0$, we have $\kappa(0) = 1$, meaning the function is well-conditioned near zero.
- At $x = \pm\pi/2$, the condition number attains its maximum:

$$\max_{x \in [-\pi/2, \pi/2]} \kappa(x) = \frac{\pi}{2}$$

- Since $\frac{\pi}{2} \approx 1.57$, the function remains numerically stable across the entire interval.

Original Algorithm Issues And Improvement

- When compute the function

$$f(x) = \frac{1 - \cos(x)}{\sin(x)}$$

- In numerical view, as $x \to 0$ (close to but not equal):
  - $1 - \cos(x)$ is very small, leading to cancellation errors.
  - $\sin(x)$ is also small, increasing numerical instability.
- Algorithm Reformulation

  Using the trigonometric identity:

$$1 - \cos(x) = 2\sin^2(x/2)$$

  we rewrite:

$$f(x) = \frac{2\sin^2(x/2)}{\sin(x)}$$

Python Implementation

```
import math

def f(x):
    if x == 0:
        return 0
    elif 0 < x <= math.pi/2 or -math.pi/2 <= x < 0:
        return 2 * (math.sin(x / 2) ** 2) / math.sin(x)
```